Lecture 31.15: DBSCAN Clustering (Code) — Scikit-Learn Implementation











📌 Topic Overview

Practical implementation of DBSCAN in scikit-learn. Demonstrates feature standardization with StandardScaler, configuring eps and min_samples, inspecting cluster labels, counting noise outliers (-1), and scatter plot visualization.

📋 Key Learning Objectives

- Importing DBSCAN from sklearn.cluster.
- Standardizing features with StandardScaler (essential for fixed eps distance radius).
- Configuring hyperparameters: eps=0.5 and min_samples=5.
- Executing fit_predict(X_scaled) to generate cluster labels and noise tags.
- Extracting noise points (label == -1) and visualizing cluster boundaries.

🔍 In-Depth Conceptual Breakdown

- **Importing Tools**: `from sklearn.cluster import DBSCAN` aur `from sklearn.preprocessing import StandardScaler` use kiya gaya.
- **Scaling is Mandatory**: DBSCAN mein ϵ ek fixed geometrical distance hota hai. Agar features scaled nahi honge, toh unscaled column radius ko distort kar dega!
- **DBSCAN Setup**: `dbscan = DBSCAN(eps=0.5, min_samples=5)` instantiate kiya gaya.
- **Fitting & Labels**: `labels = dbscan.fit_predict(X_scaled)` execute kiya gaya. Jo points cluster ke part bane unhe `0, 1, 2` mila, aur jo outliers the unhe `-1` mila.

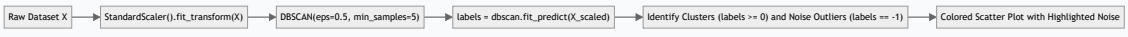
- **Noise Analysis**: `n_noise = list(labels).count(-1)` se total noise points count kiye gaye.
- **Visualization**: Seaborn scatterplot par clusters ko colorful aur noise points (-1) ko distinct black/grey markers ke sath plot kiya gaya.
- **Code Lecture**: Is lecture ka complete runnable notebook lecture_31_15.ipynb mein available hai.

📋 DBSCAN Hyperparameters in Scikit-Learn

Hyperparameter	Default Value	Tuning Rule of Thumb	Impact on Clustering
`eps`	`0.5`	K-distance elbow plot / domain distance	Too small: most points become noise; Too large: clusters merge into one
`min_samples`	`5`	Generally $2 \times D$ (where D = features)	Higher value filters more noise and demands denser regions
`metric`	`'euclidean'`	`'euclidean'`, `'manhattan'`, `'cosine'`	Distance metric to calculate neighborhood radius

---

📊 Scikit-Learn DBSCAN Execution Workflow



🔢 Mathematical Formulation

Cluster count and noise identification formula in Python:

$$K = |\{c \in \text{set}(\text{labels}) \mid c \neq -1\}|, \quad N_{\text{noise}} = \sum_{i=1}^{N} \mathbb{I}(\text{labels}_i = -1)$$

💻 Implementation

The standalone verified code is provided in the synchronized Jupyter Notebook.